
Sylvitect-core

Архитектура и алгоритмы

Техническая документация

Автоматическое формирование плана озеленения по топографической подоснове DXF: распознавание объектов, восстановление геометрии, расчёт ограничений, размещение растений и объяснение решений.

Статус документа. Формулы ниже описывают действующую реализацию в репозитории на 27 сентября 2026 года. Параметры правил и моделей читаются из конфигурации и базы данных; приведённые численные примеры служат для пояснения, а не заменяют запись конкретного прогона.

Содержание

1	Назначение и границы решения	2
2	Модель данных и поток обработки	2
3	Единицы измерения и геометрическая основа	3
4	Распознавание и восстановление инженерных сетей	4
4.1	Классификация графических примитивов	4
4.2	Короткие разрывы и неоднозначность	4
5	Правила и допустимые зоны	5
6	Планировщик посадок	6
7	Интерпретируемость и независимая проверка	7
8	Развёртывание и воспроизведение	8
9	Методы, библиотеки и модели	9
10	Ограничения и контроль качества входа	9

1. Назначение и границы решения

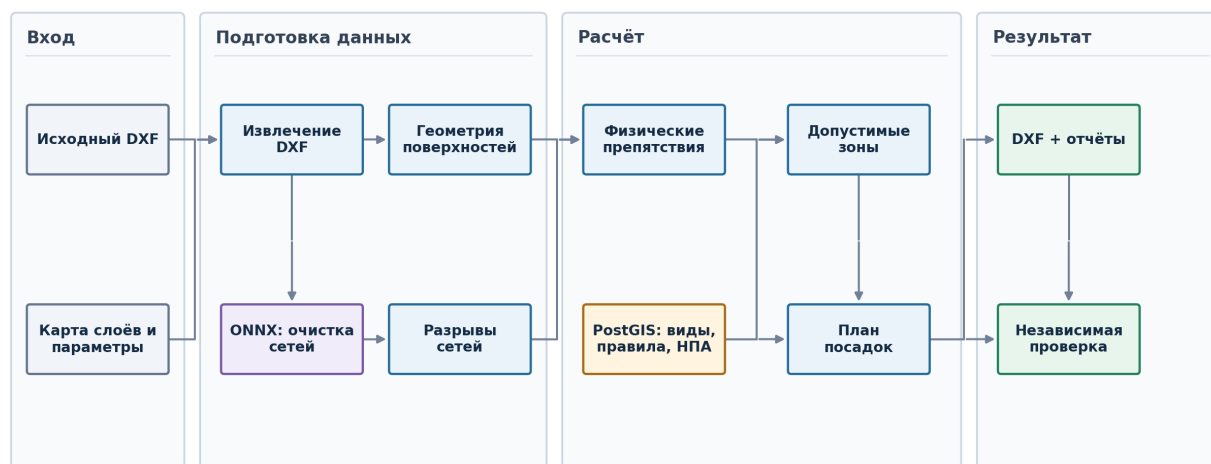
Sylvitect-core получает топографический DXF, выделяет рабочую территорию, пригодные поверхности, дороги, здания и инженерные сети, затем строит план деревьев, кустарниковых массивов и травянистого покрытия. Результат включает новый DXF с посадками, геометрические слои, отчёты о проверках и объяснения для каждой принятой посадки. Исходный чертёж сохраняется отдельно.

В MVP входят: извлечение DXF и блоков, проверка единиц, нормализация геометрии, четыре ONNX-классификатора сетей (вода, газ, теплосеть, силовой кабель), геометрическое восстановление допустимых разрывов, построение базовой и видовых зон, подбор растений, планирование посадок, DXF/PDF-выгрузка, диагностика и независимая проверка результата. Основной интерфейс — CLI и скрипты; развёртывание — Docker Compose.

Следующие релизы: расширение проверок на все 20 улиц, увеличение размеченной выборки по сетям, интерактивная правка плана с мгновенной перепроверкой, полные правила для канализации, водостока и связи, устойчивый импорт нестандартных DWG/XREF и геопривязка по подтверждённым опорным точкам. Воздушные ЛЭП со стрелочными обозначениями пока требуют проверки исходных условных знаков и трассы.

2. Модель данных и поток обработки

Единица учёта после извлечения — объект с типом, исходным слоем, геометрией и идентификатором. Промежуточные объекты хранятся в JSONL/GeoJSONL. База PostGIS содержит каталог растений, правила и нормативные ссылки. Для конкретного прогона фиксируются входной DXF, карта слоёв, масштаб единиц, выбранная модель, настройки планировщика и отчёты.

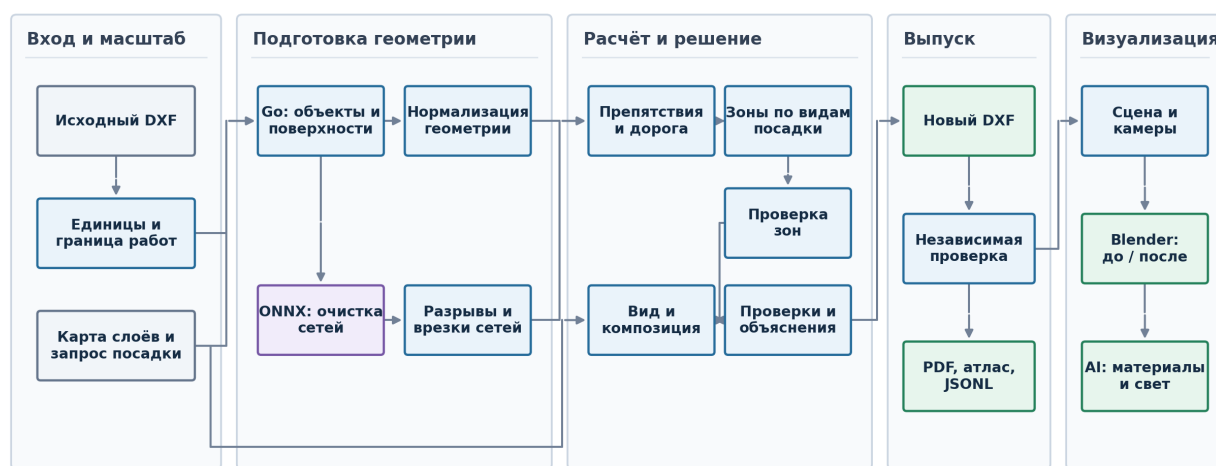


Сплошная стрелка — передача файла или результата расчёта. PostGIS поставляет справочники и правила.

Рис. 1: Компоненты и обмен данными. Подписи на схеме описывают роль каждого блока в расчёте.

Стадия	Вход и действие	Выход / контроль
Единицы и DXF	Чтение \$INSUNITS, извлечение modelspace и вложенных блоков, поставление слоёв	dxs_units_report.json, extracted_objects.jsonl

Стадия	Вход и действие	Выход / контроль
Подготовка	Нормализация поверхностей, дорог и препятствий; классификация и восстановление сетей	normalized_objects.geojsonl, cleaned_utilities.geojsonl, reconstructed_utilities.geojsonl
Ограничения	Физические исключения, буферы правил, контроль покрытия исходными данными	constraint_map.geojsonl, plant_allow_zones.geojsonl
План	Выбор вида, композиции, кандидатов и их проверка	planting_plan.geojsonl, planting_decisions.geojsonl
Выпуск	Добавление слоёв к копии чертежа, независимая верификация	result_with_planting_plan.dxf, verification_report.json, PDF



Визуализация запускается отдельно: Blender читает готовый план, AI при выборе обрабатывает кадры. DXF и координаты не изменяются.

Рис. 2: Путь от исходного DXF к проверенному чертежу и дополнительной визуализации.

3. Единицы измерения и геометрическая основа

Обозначим через q число единиц DXF в одном метре. Для расстояния правила d_m и площади A_{DXF} применяется

$$d_{DXF} = q d_m, \quad A_{m^2} = \frac{A_{DXF}}{q^2}. \quad (1)$$

Масштаб определяется из заголовка DXF или явно подтверждённого параметра запуска. При неизвестных единицах расчёт останавливается: отступы в метрах без масштаба нельзя достоверно перенести в чертёж.

Пусть W — полигон границы работ, S — объединение подтверждённых пригодных поверхностей. Стартовая поверхность C определяется так:

$$C = \begin{cases} W \cap S, & S \neq \emptyset, \\ W, & S = \emptyset \text{ (резервный режим с предупреждением)}. \end{cases} \quad (2)$$

Пусть E — объединение физических препятствий: дорог, тротуаров, твёрдых покрытий, зданий, пятен колодцев и камер теплосети. **Базовая зона** B — пространство, в котором после вычитания этих объектов остаётся поверхность для дальнейшего анализа:

$$B = \text{Polygonal}(C \setminus E). \quad (3)$$

Это ещё не разрешение на конкретное дерево или кустарник: сетевые и объектные отступы применяются на следующем шаге. В резервном режиме площадь W не считается подтверждённой пригодной поверхностью; статус и диагностические проверки сохраняют эту неопределённость.

Реализация: `src/geometry/constraint_builder.py`; `src/geometry/sidewalk_geometry.py`.

4. Распознавание и восстановление инженерных сетей

4.1. Классификация графических примитивов

В слое сети могут находиться оси труб, стрелки, выноски и подписи. Для каждого линейного примитива строится 19-мерный вектор признаков x_i : длина, прямолинейность, габариты, число вершин, степень связности концов, продолжения, соседние штрихи и характеристики связной компоненты. Отдельная модель M_t для типа сети t возвращает $p_i = M_t(x_i)$.

$$\text{decision}(p_i) = \begin{cases} \text{accepted}, & p_i \geq \tau_{a,t}, \\ \text{manual_review}, & \tau_{r,t} \leq p_i < \tau_{a,t}, \\ \text{rejected}, & p_i < \tau_{r,t}. \end{cases} \quad (4)$$

Пороги $\tau_{a,t}$ и $\tau_{r,t}$ хранятся вместе с ONNX-моделью в JSON-метаданных; вектор подаётся в заданном там порядке как `float32`. При обучении пространственные группы позволяют отделять участки проверки от обучающих. Для кабеля порог принятия ориентирован на точность, для остальных сетей — на полноту. Валидационные показатели относятся к размеченным выборкам и не гарантируют такую же точность на новой улице.

Алгоритм 1. Очистка сети

1. Извлечь из целевых слоёв линейные примитивы и построить их признаки с учётом соседей.
2. Выполнить ONNX-инференс отдельно для воды, газа, теплосети и кабеля.
3. Разделить примитивы по двум порогам; сохранить принятые оси и отдельный слой ручной проверки.
4. Передать в восстановление только принятую геометрию и сохранить исходные ID для трассировки.

Реализация: `src/detection/utilities/detector.py`;
`src/detection/utilities/onnx_model.py`; `models/utility_detector/latest/metadata.json`.

4.2. Короткие разрывы и неоднозначность

Открытые концы принятых линий индексируются пространственно. Для кандидата продолжения длиной g вычисляются два угла α_1, α_2 между направлением линии у концов и соединяющим вектором. Кандидат проходит геометрический фильтр $g \leq g_{\max}$ и $\alpha_1, \alpha_2 \leq \alpha_{\max}$. Его базовый скоринг:

$$D = 1 - \frac{\alpha_1 + \alpha_2}{2\alpha_{\max}}, \quad L = 1 - \frac{g}{g_{\max}}, \quad (5)$$

$$s_{\text{cont}} = 0.7D + 0.3L. \quad (6)$$

Для присоединения конца к внутренней точке соседней линии используется проекция и весовая схема $s_{\text{junction}} = 0.65D + 0.35L$; в этом случае D зависит от одного угла. Настройки содержат лимиты разрыва, углов и пороги решений. При параллельных трассах поддержка соседним сходным продолжением повышает оценку. Если несколько кандидатов конкурируют, алгоритм

фиксирует неоднозначность и может оставить ветвление по настроенной политике. Принятая соединительная линия записывается отдельно от исходной оси, что позволяет увидеть именно реконструкцию. Для силового кабеля автоматическое соединение разрывов отключено.

Алгоритм 2. Восстановление фрагмента сети

1. Найти свободные концы принятой сети и локальные кандидаты через пространственный индекс.
2. Отсечь дальние и геометрически несовместимые пары; оценить продолжения и присоединения.
3. Учесть параллельность, конкурирующие соединения и заданные пороги.
4. Записать принятые соединители, сомнительные кандидаты и причины решения в отдельные артефакты.

Реализация: `src/detection/network_reconstructor.py`;
`src/core/network_reconstructor_config.yaml`.

5. Правила и допустимые зоны

Для типа посадки $t \in \{\text{tree, shrub}\}$ активные правила R_t задают целевой объект G_r , отступ d_r в метрах и ссылку на НПА. Геометрия запрета для правила и результирующая **допустимая зона** записываются как

$$X_r = \text{Buffer}(G_r, qd_r), \quad A_t = \text{Polygonal} \left(B \setminus \bigcup_{r \in R_t} X_r \right). \quad (7)$$

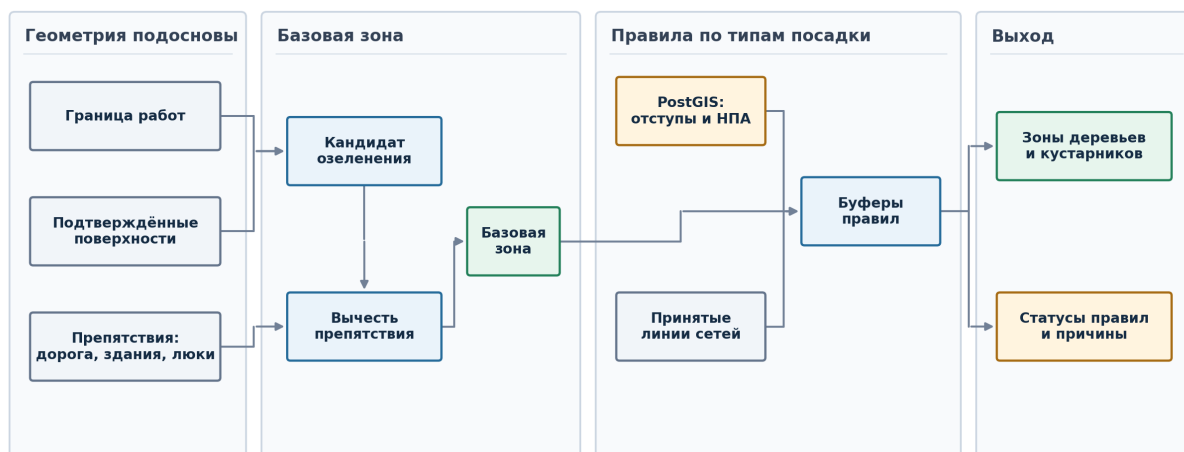
Таким образом, B отвечает на вопрос «где в принципе есть поверхность», а A_t — «где по доступным данным разрешён центр посадки данного типа». Условия размещения кроны, расстояния между растениями и выбранного вида проверяются затем при генерации плана. Для травянистого покрытия отдельная видовая зона правил не строится: оно занимает оставшуюся часть B с учётом существующих массивов и уже размещённых элементов.

В текущем каталоге заданы следующие минимальные отступы от оси ствола или центра посадки до объекта. Их источник в каталоге — [СП 42.13330.2026, таблица 6.3](#). Для конкретного прогона действует содержимое базы после выполнения `seed.py`.

Объект	Дерево, м	Кустарник, м	Код правила (пример)
Здание	5	1.5	TREE_BUILDING_5
Тротуар	0.7	0.5	TREE_SIDEWALK_0_7
Край проезжей части	2	1	TREE_ROAD_EDGE_2
Газопровод	1.5	—	TREE_GAS_1_5
Теплосеть	2	1	TREE_HEAT_2
Водопровод	2	—	TREE_WATER_2
Силовой кабель	2	0.75	TREE_POWER_CABLE_2

Прочерк означает отсутствие отдельного правила данного типа в текущем каталоге; он не является утверждением о нормативной допустимости посадки.

Если объект правила отсутствует, правило получает статус `unavailable`; вычитание по нему не выполняется, а неполнота данных попадает в отчёт. Если сеть имеет сомнительный источник или неполную геометрию, статус зоны указывает на необходимость проверки. Проверка результата отличает вычисленное отсутствие нарушения от отсутствия данных о препятствии.



Если объект правила отсутствует, порог остаётся в отчёте; отсутствующая геометрия не образует нулевой буфер.

Рис. 3: Базовая область, буферы правил и зоны для разных типов посадок.

Пример: при $q = 1$ и правиле «кустарник — теплосеть: 1 м» каждый сегмент подтверждённой теплотрассы расширяется буфером шириной 1 м, после чего буфер вычитается из B . Фактическое значение, код правила и ссылка на пункт нормативного документа берутся из записи PostGIS, а не из этой иллюстрации. Для камер теплосети действует самостоятельное физическое исключение контура.

Реализация: `src/rules/plant_allow_zone.py`; `scripts/seed.py`.

6. Планировщик посадок

По умолчанию генератор строит несколько целых схем на участок, а OR-Tools CP-SAT выбирает совместимое сочетание схем. Для каждой схемы i вводится переменная $x_i \in \{0, 1\}$; n_i означает число растений, q_i — оценку вместимости и композиции. Модель максимизирует $\sum_i q_i x_i$ при ограничениях: не более одной схемы на участок, $\sum_i n_i x_i \leq N$, а для конфликтующих схем $x_i + x_j \leq 1$. Конфликт определяется геометрической проверкой расстояний до запуска решателя.

Деревья размещаются первыми; кустарники учитывают их положение. Вид растения задаётся профилем. Оптимальность относится к перечисленным вариантам и заданной оценке. Ограничение времени может дать допустимое решение без доказанной оптимальности; статус и статистика сохраняются в `layout_optimizer_runs`. Все выбранные точки проходят повторную проверку правил. Подробнее: `docs/cp_sat_layout.md`. Планировщик читает допустимые зоны, каталог морозостойких растений и запрос на композицию. Профиль вида содержит тип, площадь/радиус пятна, шаг между растениями, расстояние до других посадок и причины выбора. Деревья получают точечные центры; кустарник может описываться непрерывным полигоном массива; травянистое покрытие — полигоном остатка. Выбор вида и композиции фиксируется в объяснении.

Для двух профилей i, j минимальный шаг центров в метрах равен

$$s_{ij} = \begin{cases} \max(s_i, s_j, r_i + r_j), & t_i = t_j, \\ \max(a_i, a_j, r_i + r_j), & t_i \neq t_j, \end{cases} \quad (8)$$

где s_i — шаг вида, a_i — избегаемое расстояние до другого типа, r_i — радиус пятна. Перед принятием точки проверяется её нахождение в безопасной области с учётом радиуса, существующих деревьев и уже выбранных посадок.

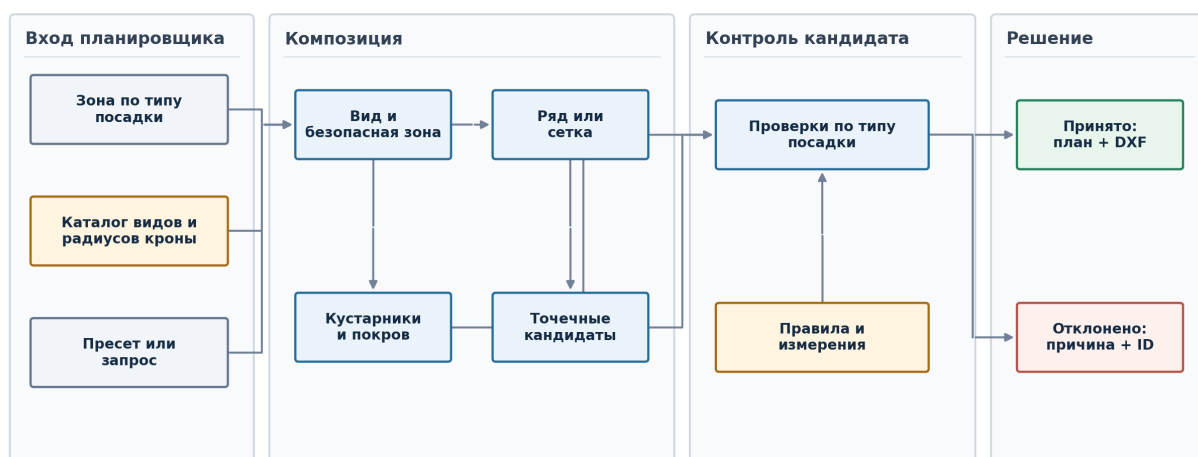
Для площадного заполнения используется сдвинутая по соседним строкам сетка. При шаге s расстояние между строками $h = \sqrt{3}s/2$; каждая вторая строка сдвигается на $s/2$. Для вытяну-

тых фрагментов сначала оценивается продольная ось, затем несколько фаз ряда. В линейном режиме допустимы схемы с числом принятых точек не менее $\lceil 0.9N_{\max} \rceil$; из них выбирается композиция с большим числом соседних пар рядов, меньшим числом одиночных рядов и лучшим балансом краёв. Это даёт ровную посадку без существенной потери вместимости.

Алгоритм 3. Размещение и протокол решений

1. Выбрать вид и тип композиции для каждой части зоны; сначала обрабатывать более крупные пятна.
2. Сгенерировать фазы точечной сетки или линейных рядов; проверить каждую точку по зоне, соседям и существующей растительности.
3. Записать принятые точки в план, отклонённые — в диагностику с фактическим расстоянием и требуемым пределом.
4. Сформировать кустарниковые полигоны и остаточное травянистое покрытие. Для массива оценить количество кустов как $\lceil A/(s^2\sqrt{3}/2) \rceil$ при положительном s .
5. Сохранить выбранную фазу, число точек и список причин принятия или отклонения для последующей проверки.

Реализация: `src/planting/placement_generator.py`; `src/planting/service.py`.



Точка дерева и полигон кустарникового массива — разные выходные типы; каждый получает проверку и ID.

Рис. 4: Построение композиций и проверка кандидатов.

7. Интерпретируемость и независимая проверка

Каждая принятая посадка получает ID, геометрию, выбранный вид, источник зоны, список пройденных проверок, фактические и требуемые расстояния, код правила и нормативную ссылку с пунктом. Для отклонённого кандидата сохраняется проверка, вызвавшая отказ, и измеренное значение. Подробности распределены по трём файлам:

- `planting_plan.geojsonl` — принятый план;
- `planting_decisions.geojsonl` — решения по кандидатам;
- `planting_explanations.md` — читаемые пояснения.

PDF даёт краткое представление для эксперта. Ссылка на НПА берётся из версии правила в базе, поэтому при изменении правил пересчёт создаёт новый набор объяснений.

Интерпретация решения c может быть представлена записью

$$E(c) = \{ID, geometry, species, checks, measurements, rule_ids, references, source_quality\}. \quad (9)$$

Это структура отчёта, а не дополнительная формула принятия: итоговый статус определяется последовательностью проверок и записывается вместе с ними. Для экспертной оценки важно открывать полный протокол, если краткий PDF скрывает часть кандидатов.

Алгоритм 4. Контроль готового результата

1. Повторно считать геометрию плана и допустимых зон из сохранённых артефактов.
2. Проверить принадлежность центров зонам, непересечение с физическими препятствиями и минимальные расстояния между посадками.
3. Сопоставить исходные объекты с копией DXF и проверить структуру выходного файла.
4. Выдать passed только для набора выполненных вычислимых проверок; недостаток входных данных оставить явным в отчёте.

Реализация: `scripts/verify_outputs.py`; `src/planting/service.py`.

8. Развёртывание и воспроизведение

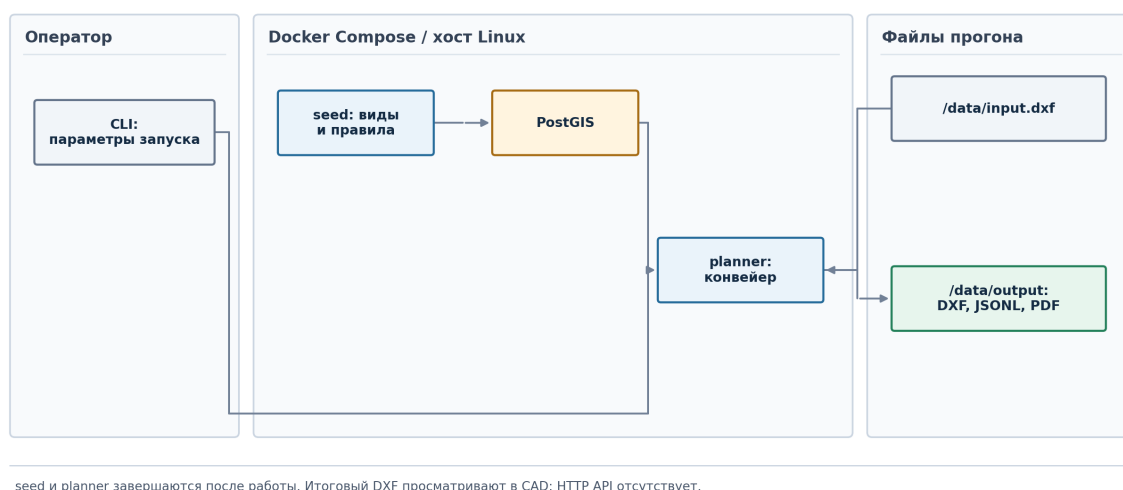


Рис. 5: Развёртывание: контейнер планировщика, PostGIS и каталог входных/выходных данных.

На Linux или MocTex.OC с Docker Engine и Compose из корня репозитория выполнить:

```
mkdir -p docker-data
cp /path/to/input.dxf docker-data/input.dxf
docker compose build planner
docker compose up -d --wait postgis
docker compose --profile pipeline run --rm planner \
  /data/input.dxf /data/output --mode full
```

Затем открыть отчёт `verification_report.json` в `docker-data/output`. При статусе `passed` следует посмотреть `result_with_planting_plan.dxf` и PDF-отчёт в CAD и просмотрщике. Проверка на целевой сборке MocTex.OC после последнего рефакторинга ещё должна быть зафиксирована отдельным прогоном. На Windows установку выполняет `scripts/install.ps1`, а полный

запуск — `scripts/run_pipeline.ps1` с `-InputDxf`, `-OutputDirectory` и `-PipelineMode full`. Повторный запуск в новой выходной папке сохраняет результаты предыдущего расчёта.

HTTP-бэкенда у MVP нет; способы взаимодействия — CLI, конфигурационные файлы и выходные артефакты. OpenAPI/Swagger потребуется при появлении HTTP API.

9. Методы, библиотеки и модели

Компонент	Назначение и источник
Go-парсер DXF ezdxf Shapely / GEOS	Потоковое извлечение объектов и блоков; Go , yaml.v3 . Анализ DXF, нормализация и создание выходного чертежа. Буферы, пересечения, разности, пространственные индексы, полигонизация.
PostGIS OR-Tools CP-SAT	Хранение правил, видов растений и нормативных ссылок. Выбор совместимых схем посадок с ограничением количества и расстояний.
scikit-learn	Обучение четырёх бинарных RandomForest-классификаторов.
ONNX / ONNX Runtime	Сериализация моделей и инференс в конвейере. Метаданные с порогами и порядком признаков хранятся рядом с ONNX.
ReportLab	Формирование PDF-отчёта по посадкам; Blender — дополнительные изображения до/после.

10. Ограничения и контроль качества входа

Чертежи улиц пока не имеют единого стандарта слоёв и условных обозначений. Часть объектов находится во вложенных блоках, отдельные DXF возникают из DWG с отсутствующими внешними ссылками или ошибками таблиц CAD. Поэтому карта слоёв и отчёт извлечения должны проверяться на новой улице; ошибочный или пустой источник нельзя считать доказательством свободной территории.

Неполные сведения о подземной сети, воздушной ЛЭП или напряжении требуют отдельной проверки. Стрелочные знаки ЛЭП сами по себе не задают однозначную трассу и охранную зону. Геопривязка локальной системы к WGS84 невозможна без достоверных контрольных точек и определения исходной системы координат. Автоматическая проверка покрывает только описанные и доступные на чертеже ограничения; итоговый проект подлежит инженерной экспертизе.

При переносе на новую улицу минимальная проверка включает: отчёт единиц, число объектов по ключевым типам, положение слоёв относительно границы работ, диагностические слои очищенных сетей и исключений, отчёт зон, протокол отклонённых кандидатов, открытие DXF в целевом CAD и результат `verification_report.json`.

Исходный код и настройки являются источником точных параметров конкретного прогона. Документ объясняет метод и точки проверки; численные значения действующих правил определяются сохранённой версией каталога PostGIS и метаданными модели.